

Section 6: Evaluation and Monitoring (12%) - Study Guide

Sources confirmed from multiple databricks.com and mlflow.org pages for MLflow evaluation, inference tables, Agent Monitoring, AI Gateway usage tracking, and custom scorers. Verify at docs.databricks.com using the search terms provided at the end of each section.

Terminology Breakdown

- **Evaluation (offline):** The process of assessing an LLM application's quality *before* deployment, using curated test datasets and automated scorers to answer "Is this ready to ship?"
- **Monitoring (online):** The process of tracking a deployed LLM application's behavior in production *after* deployment, using live traffic data to answer "Is this still performing well?"
- **Ground Truth:** A known, correct reference answer for a given input. Used as the benchmark to measure how accurately a model's output matches the expected response.
- **Golden Dataset / Evaluation Dataset:** A curated collection of representative inputs paired with expected outputs (ground truth) used to reliably measure model performance over time.
- **LLM-as-a-Judge:** A technique where a second, capable LLM is used to evaluate the quality of outputs from the primary LLM, assigning scores on dimensions like relevance, groundedness, or tone.
- **Scorer:** In MLflow evaluation, a scorer is a function (either a built-in judge, a custom LLM judge, or a code-based Python function) that assigns a numerical score or label to a model's output for a given input.
- **Groundedness:** A metric that measures whether the model's answer is supported by the retrieved context documents. A grounded answer contains no claims beyond what is stated in the provided context. Does NOT require ground truth.
- **Relevance to Query:** A metric that measures whether the model's response directly addresses the user's question. Does NOT require ground truth.
- **Answer Correctness:** A metric that compares the model's answer to a known correct answer. **Requires ground truth.**
- **Answer Similarity:** A metric measuring semantic closeness between the model's output and a reference answer. **Requires ground truth.**
- **Retrieval Precision / Recall@k:** Metrics measuring whether the retrieved chunks actually contain the information needed to answer the query. Used to evaluate the retrieval step of a RAG pipeline.
- **MLflow Tracing:** A feature that captures every step of an agent's execution — LLM calls, tool invocations, retrieved documents, intermediate outputs — as a structured, inspectable record with timing data.

- **Inference Tables:** Delta tables in Unity Catalog that automatically log every input and output from a Databricks Model Serving endpoint. The primary tool for production monitoring.
 - **Agent Monitoring:** The Databricks capability (part of Mosaic AI) that provides continuous quality tracking of live deployed agents using inference tables and LLM judges applied to production traffic.
 - **Unity AI Gateway Usage Tables:** System tables (e.g., `system.serving.endpoint_usage`) populated by the Unity AI Gateway that track token consumption, request counts, and cost per endpoint, user, or team.
 - **Rate Limiting:** A Unity AI Gateway feature that caps the number of requests or tokens per time window per user or Service Principal, preventing runaway costs and ensuring fair resource allocation.
 - **Token Budget:** An organizational control setting a maximum number of tokens (input + output) allowed per endpoint, team, or project within a billing period.
 - **Provisioned Throughput:** A billing mode for Databricks Foundation Model APIs where you reserve dedicated token-per-second capacity and pay hourly regardless of utilization — optimized for predictable production loads.
 - **Pay-per-token:** A billing mode for Databricks Foundation Model APIs where you pay only for tokens consumed — optimized for variable or prototype workloads.
 - **SME (Subject Matter Expert):** A domain expert (e.g., a doctor, lawyer, or product specialist) whose qualitative judgment is used to validate that an AI agent's responses are accurate and appropriate for a specific field.
 - **MLflow Review App:** A web-based UI that allows SMEs and human evaluators to test agents interactively, rate responses, and annotate production traces — without writing any code.
 - **Assessment (MLflow):** The stored record of a human evaluator's feedback on a specific trace. Contains ratings, corrections, and optional "expected output" labels.
 - **MMLU (Massive Multitask Language Understanding):** An academic benchmark testing a model's knowledge and reasoning across 57 subjects (math, law, ethics, science, etc.) using multiple-choice questions. Used for initial model selection, not domain-specific evaluation.
 - **HumanEval:** An academic benchmark specifically measuring coding ability (Python function generation). Useful for selecting code-generation models.
 - **MT-Bench:** An academic benchmark evaluating multi-turn conversational ability and instruction following. Used to assess chat model quality.
 - **TTFT (Time to First Token):** A latency metric measuring the time from when a request is sent until the model begins streaming its first output token. Critical for user-facing real-time applications.
 - **Throughput (tokens/second):** The number of tokens an endpoint can process per second. Key metric for batch inference workload sizing.
-

1. Select an LLM based on quantitative metrics generated in experiments

Concept: Rather than choosing a model by intuition or brand name, you run controlled experiments comparing multiple models head-to-head and select based on measured, objective results relevant to your specific use case.

Databricks Context & Implementation:

Step 1 — Academic Benchmarks (Initial Shortlisting): Use industry benchmarks to create a shortlist of candidate models. Know what each benchmark measures:

Benchmark	What It Tests	When Relevant
MMLU	General knowledge, reasoning across 57 subjects	General-purpose enterprise assistant
HumanEval	Code generation (Python functions)	Code generation / SQL / data engineering tasks
MT-Bench	Multi-turn chat quality, instruction following	Conversational / RAG chatbots
GSM8K	Multi-step math reasoning	Finance, analytics calculations
MATH	Advanced math problem-solving	Specialized scientific / quantitative tasks

 **Warning:** High benchmark scores do NOT guarantee success on your specific enterprise data. Academic benchmarks are only a starting filter, not a final decision.

Step 2 — Domain-Specific MLflow Experiments: For each shortlisted model, run `mlflow.evaluate()` against your own golden dataset with domain-specific scorers and system performance metrics:

```

import mlflow
from mlflow.genai.scorers import RelevanceToQuery, RetrievalGroundedness

# Evaluate model A and model B as separate MLflow runs
for model_uri, model_name in [("model_a_uri", "llama-3-70b"), ("model_b_uri", "mixtral-8x7b")
    with mlflow.start_run(run_name=model_name):
        results = mlflow.evaluate(
            model=model_uri,
            data=golden_dataset,
            scorers=[RelevanceToQuery(), RetrievalGroundedness()]
        )

```

Key metrics to compare in the MLflow Experiment UI:

Metric	Type	Meaning
groundedness	Quality	Are answers supported by context?
relevance_to_query	Quality	Do answers address the question?
answer_correctness	Quality (needs GT)	Are answers factually accurate?
latency_ms / ttft_ms	Performance	User-perceived response speed
tokens_per_second	Performance	Throughput for batch use cases
cost_per_1k_tokens	Cost	Total economic cost of inference

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "MLflow evaluate generative AI scorers" * "Foundation Model APIs model comparison"

2. Use MLflow and Agent Framework for scoring and tracing

Concept: MLflow's tracing and evaluation capabilities are the core instrumentation layer for understanding what your agent did, why it made each decision, and whether the result was good.

Databricks Context & Implementation:

Tracing — Capturing the "What and How":

```
import mlflow

# Option 1: Enable automatic tracing for LangChain agents
mlflow.langchain.autolog()

# Option 2: Use the @mlflow.trace decorator for custom code
@mlflow.trace(span_type="RETRIEVER")
def retrieve_documents(query: str):
    return vector_search_index.similarity_search(query, num_results=5)

# Option 3: Use the Mosaic AI Agent Framework agents.deploy()
# – tracing is enabled automatically
```

Each trace captures a full **span tree**: the top-level request, sub-spans for each LLM call, tool invocations, retrieved documents, and their timings. Viewable in the MLflow UI under Experiments → Traces.

Scoring — Measuring "How Well":

```
from mlflow.genai.scorers import RelevanceToQuery, RetrievalGroundedness, Safety

results = mlflow.genai.evaluate(
    data=eval_dataset,          # List of dicts with "inputs" and optionally "expected_out
    predict_fn=my_agent_function, # Your agent callable (or pass existing traces)
    scorers=[
        RelevanceToQuery(),      # No ground truth needed
        RetrievalGroundedness(), # No ground truth needed
        Safety()                 # No ground truth needed
    ]
)
```

What the output contains: * Per-row scores for each scorer * Aggregate statistics (mean, min, max per scorer) * Drill-down links to the trace for each failing row

Where to verify in Databricks Docs: Search docs.databricks.com for: * "mlflow.genai.evaluate scorers" * "MLflow Tracing agent Databricks"

3. Use inference logging to assess deployed RAG application performance

Concept: Once your RAG application is deployed to a Databricks Model Serving endpoint, inference tables automatically capture every request and response as a permanent, queryable record in Unity Catalog —

enabling production-time performance assessment.

Databricks Context & Implementation:

Enable inference tables (UI): 1. Navigate to your Model Serving endpoint in the Databricks UI. 2. Click the **Inference tables** section → **Set Up**. 3. Choose the Unity Catalog catalog and schema for log storage. 4. Databricks automatically creates a Delta table and begins logging all traffic.

What gets logged: * Full request payload (user query, retrieved context chunks passed to LLM) * Full response payload (the model's generated answer) * Timestamp, latency, endpoint version * MLflow trace data (if tracing is enabled on the endpoint)

Querying inference logs to assess performance:

```
-- Find slowest queries in the last 7 days
SELECT request_id, latency_ms, status_code
FROM main.monitoring.my_rag_endpoint_payload_logs
WHERE date >= CURRENT_DATE - 7
ORDER BY latency_ms DESC
LIMIT 100;

-- Find all requests where the model returned an error
SELECT request_id, request, response
FROM main.monitoring.my_rag_endpoint_payload_logs
WHERE status_code != 200;
```

Continuous quality monitoring: After collecting inference table data, apply LLM judges retroactively to production traces — using `mlflow.evaluate()` on the logged data — to track groundedness and relevance scores over time and detect quality drift.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Inference tables Databricks Model Serving" * "Enable inference tables serving endpoint"

4. Use Databricks features to control LLM costs

Concept: LLM costs can escalate quickly. Databricks provides layered controls — at the gateway, billing, and infrastructure level — to keep spending predictable.

Databricks Context & Implementation:

Control Lever 1 — Unity AI Gateway Rate Limiting: Set token-based or request-based quotas per user, Service Principal, or team. When a limit is hit, the gateway returns HTTP 429 (Too Many Requests) rather than charging for overages.

Gateway Configuration → Rate Limits:

- Per user: 50,000 tokens/hour
- Per endpoint: 500,000 tokens/day

Control Lever 2 — System Tables for Cost Attribution: Query Databricks system tables to understand where tokens are being consumed:

```
-- Token usage by endpoint over the last 30 days
SELECT endpoint_name,
       SUM(total_tokens) AS total_tokens,
       SUM(total_tokens) * 0.0001 AS estimated_cost_usd -- example per-token rate
FROM system.serving.endpoint_usage
WHERE timestamp >= CURRENT_TIMESTAMP - INTERVAL 30 DAYS
GROUP BY endpoint_name
ORDER BY total_tokens DESC;
```

Control Lever 3 — Right-Sizing Model Selection: A 7B parameter model often costs 10–20× less per token than a 70B model. For tasks where the smaller model performs acceptably (e.g., classification, extraction), switching saves significant cost. Use MLflow experiments to validate that quality trade-offs are acceptable.

Control Lever 4 — Provisioned Throughput Autoscaling: For Provisioned Throughput endpoints, configure a min/max token-per-second range rather than a fixed value. The endpoint autoscales within that range — you pay for idle capacity at the minimum, not at the maximum:

Provisioned Throughput Configuration:

- Min tokens/s: 1,000 (baseline cost)
- Max tokens/s: 10,000 (scales up during peak)

Control Lever 5 — Batch Inference with `ai_query()` : For non-real-time workloads (e.g., nightly enrichment), use batch `ai_query()` instead of real-time endpoints. Batch inference is generally more cost-efficient for large-scale processing.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Unity AI Gateway rate limiting" * "system.serving.endpoint_usage Databricks"

5. Use inference tables and Agent Monitoring to track a live LLM endpoint

Concept: Inference Tables capture raw traffic data; **Agent Monitoring** builds on top of this by automatically applying quality scorers (LLM judges) to the logged production traces — creating a

continuous quality dashboard without manual intervention.

Databricks Context & Implementation:

Infrastructure Monitoring (Serving UI dashboards): The Model Serving endpoint UI provides built-in infrastructure metrics: * **Request latency** (P50, P95, P99) — tracks slowdowns * **Error rate** — tracks 4xx/5xx spikes * **CPU/memory utilization** — indicates resource pressure * **Throughput (requests/second)** — tracks usage patterns

Payload Monitoring (Inference Tables): Captures the *content* of requests and responses as described in Objective 3. This is the raw data layer.

Quality Monitoring (Agent Monitoring): Agent Monitoring combines inference table data with automated LLM judge scoring to track *quality metrics over time*: * **Groundedness score over time** — detects if the model starts hallucinating more * **Relevance score over time** — detects if user queries are drifting away from the knowledge base * **Safety violations count** — tracks if harmful content is being generated in production

Enabling Agent Monitoring: When you deploy an agent using `agents.deploy()` from the Mosaic AI Agent Framework, Agent Monitoring is automatically configured. For custom deployments: 1. Enable inference tables on the endpoint 2. Ensure MLflow tracing is active on the endpoint 3. Use the Agent Monitoring dashboard in the Databricks UI

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Mosaic AI Agent Monitoring" * "Databricks Model Serving endpoint metrics"

6. Identify evaluation judges that require ground truth

Concept: This is a classification knowledge point — you must know exactly which MLflow built-in scorers/judges can run without a reference answer, and which cannot.

Databricks Context & Implementation:

Judges that do NOT require ground truth (input + output only):

Judge / Scorer	What It Measures	Why No GT Needed
RelevanceToQuery	Is the response relevant to the question?	Compares output to the input query only
RetrievalRelevance	Are retrieved chunks relevant to the question?	Compares retrieved context to the input query
RetrievalGroundedness	Is the answer supported by retrieved context?	Compares output to retrieved context (not a reference answer)
Safety	Is the content free from harmful material?	Evaluates content against safety policy
Guidelines	Does the response follow custom style/tone rules?	Evaluates against natural-language rules, not a reference answer

Judges that DO require ground truth (input + output + expected reference answer):

Judge / Scorer	What It Measures	Why GT Is Required
AnswerCorrectness	Is the answer factually correct?	Must compare to the known correct answer
AnswerSimilarity	How semantically close is the answer to the reference?	Must compare against the reference text
Exact Match (code-based)	Does the output exactly match expected output?	Requires the expected string to compare against

Exam tip: The key distinction is this: - **No GT needed** → judges that compare the *output to the input or the retrieved context* - **GT required** → judges that compare the *output to a known correct reference answer*

Where to verify in Databricks Docs: Search docs.databricks.com for: * "MLflow built-in scorers genai" * "MLflow evaluate ground truth required"

7. Use AI Gateway (Inference Tables, Usage Tables, and rate limiting) to track an LLM or agent deployed via Agent Framework

Concept: The Unity AI Gateway is the central control and observability plane for all LLM traffic. It provides three specific tracking capabilities testable on the exam.

Databricks Context & Implementation:

Tool 1 — Inference Tables (Payload-Level Logging): Automatically enabled for endpoints registered through the Unity AI Gateway. Logs every request/response payload as a Delta table row in Unity Catalog. Used for: * Content audit and compliance * Debugging specific failed queries * Building training data from production traffic

Tool 2 — Usage Tables (Token-Level Cost Tracking): `system.serving.endpoint_usage` (and related system tables) tracks consumption at the token level. Queryable via SQL:

```
SELECT
  endpoint_name,
  principal_name,          -- which user or service principal made the call
  SUM(input_tokens) AS input_tokens,
  SUM(output_tokens) AS output_tokens,
  SUM(total_tokens) AS total_tokens,
  COUNT(*) AS request_count
FROM system.serving.endpoint_usage
WHERE timestamp >= CURRENT_DATE - 7
GROUP BY endpoint_name, principal_name
ORDER BY total_tokens DESC;
```

Tool 3 — Rate Limiting (Traffic Control): Configured in the Unity AI Gateway UI per endpoint: * **Per-user limits:** Prevent a single user from consuming all capacity * **Per-Service-Principal limits:** Control application-level consumption * **Endpoint-wide limits:** Set an absolute ceiling on total endpoint traffic * Rate limits are enforced as **token-based quotas** (not just request counts) because LLM resource consumption scales with token length, not request volume

How they connect in the Agent Framework: When you deploy an agent with `agents.deploy()` : 1. The agent endpoint is automatically registered behind the Unity AI Gateway 2. All traffic is routed through the Gateway → inference tables are populated automatically 3. Token usage flows into system usage tables automatically 4. You configure rate limits in the Gateway UI without any application code changes

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Unity AI Gateway rate limiting" * "system.serving endpoint usage tables" * "AI Gateway payload logging"

8. Use Databricks custom Scorers for evaluating agents and LLMs

Concept: Built-in MLflow judges cover common quality dimensions, but every enterprise application has unique requirements. Custom scorers let you define exactly what "good" means for your specific use case.

Databricks Context & Implementation:

Type 1 — Code-Based Custom Scorer (deterministic): A pure Python function. Best for exact match, format validation, keyword presence, or business rule checks:

```

from mlflow.genai.scorers import scorer

@scorer
def json_format_scorer(inputs, outputs):
    """Check if the output is valid JSON."""
    try:
        import json
        json.loads(outputs["response"])
        return 1.0 # Perfect score
    except Exception:
        return 0.0 # Failed

@scorer
def keyword_presence_scorer(inputs, outputs):
    """Check if required legal disclaimers are present."""
    required = ["not financial advice", "consult a professional"]
    text = outputs["response"].lower()
    matches = sum(1 for kw in required if kw in text)
    return matches / len(required) # Partial credit

```

Type 2 — Custom LLM Judge (AI-based): Uses an LLM as the evaluator with a custom prompt. Best for domain-specific quality criteria that require reasoning:

```

from mlflow.genai.scorers import make_genai_metric

brand_tone_scorer = make_genai_metric(
    name="brand_tone",
    definition="Does the response match a professional, empathetic, and concise brand tone?",
    grading_prompt="""
Score 1 (Poor) to 5 (Excellent). A score of 5 means the response is:
- Professional (no slang)
- Empathetic (acknowledges the user's situation)
- Concise (under 100 words)
Response: {output}
Score:
""",
    model="endpoints:/databricks-meta-llama-3-70b-instruct"
)

```

Running custom scorers alongside built-in ones:

```

results = mlflow.genai.evaluate(
    data=eval_dataset,
    scorers=[
        RelevanceToQuery(),          # built-in
        json_format_scorer,          # custom code-based
        brand_tone_scorer,           # custom LLM judge
    ]
)

```

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "MLflow custom scorers genai evaluate" * "make_genai_metric Databricks"

9. Incorporate SME feedback to improve agent performance

Concept: Automated judges are powerful but limited by their training. Subject matter experts (SMEs) — doctors, lawyers, product managers — provide qualitative judgment that automated systems cannot fully replicate. Incorporating their feedback creates a virtuous improvement cycle.

Databricks Context & Implementation:

The MLflow Review App — Primary Tool: A Databricks-hosted web UI (no coding required for SMEs) with two modes: * **Interactive Chat Mode:** SME interacts with the agent directly, testing edge cases and rating responses in real-time with thumbs up/down and written comments. * **Trace Labeling Mode:** SME reviews a curated set of historical production traces, adding ratings and corrections ("expected output") to each one.

Feedback is stored as Assessments: Each piece of human feedback is stored as an **Assessment** object in MLflow, attached to the specific trace it evaluated. Assessments contain: * A rating (e.g., 1–5, thumbs up/down) * An optional explanation comment * An optional "expected output" (what the SME thinks the correct answer should have been)

Using SME feedback to improve the agent:

SME reviews traces → Assessments created



Assessments collected into new Evaluation Dataset



`mlflow.genai.evaluate()` run on new dataset



Lowest-scoring rows identified → Root cause analysis via tracing



Fix: Update prompt, swap model, adjust chunking, or add retrieved documents



Re-evaluate → Deploy improved agent

Turning SME feedback into custom scorer training data: You can use the SME-labeled assessments to fine-tune your custom LLM judge — teaching it to score responses the same way your SMEs would. This "aligns" the automated judge with domain expertise at scale.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "MLflow Review App human feedback" * "Mosaic AI Agent Evaluation SME feedback"

Putting It All Together — The Full Evaluation & Monitoring Loop

DEVELOPMENT (Offline Evaluation)

Build agent → Enable MLflow Tracing → Create golden dataset
→ Run `mlflow.genai.evaluate()` with built-in + custom scorers
→ Review results in MLflow Experiment UI
→ Fix issues → Re-evaluate → Deploy

↓ Deploy via `agents.deploy()` ↓

PRODUCTION (Online Monitoring)

Live traffic → Unity AI Gateway
→ Inference Tables (raw payload logging)
→ Usage Tables (token cost tracking)
→ Rate Limiting (cost control)
→ Agent Monitoring (quality scores over time)
→ MLflow Tracing (execution visibility)

↓ Production data feeds back ↓

IMPROVEMENT (Continuous Learning)

Inference logs → SME Review App → Assessments
→ New evaluation dataset → Re-evaluate → Improve → Re-deploy